ADR-0002 · Decisiones del Sprint 1 (dominio: clientes, facturas y autenticación)

Fecha: 2026-10-28 · Estado: aceptada

Las dos propuestas abiertas del ADR-0001 quedaron **aprobadas**: JWT + refresh rotatorio para escritorio y dashboard, y `currency` limitada a `ARS` con un CHECK.

Decisiones

#	Decisión	Motivo
1	Una sola migración inicial (`0001`) crea **todas** las tablas de §7, incluida `payments` .	El saldo y el estado dependen de los pagos; el *servicio* de pagos se implementa en el Sprint 2.
2	Tabla adicional `refresh_tokens` (solo hash SHA-256 del token).	Necesaria para rotar, revocar en logout y detectar reutilización. No figura en §7.
3	Auth: **access JWT HS256 de 15 min** + **refresh opaco de 7 días, rotatorio**. Reutilizar un refresh ya rotado revoca todas las sesiones del usuario.	§8. Mitiga el robo de tokens.
4	Contraseñas con **Argon2id** (`argon2-cffi`), mínimo 10 caracteres. Si el email no existe también se verifica contra un hash ficticio.	§8. El tiempo de respuesta no revela qué emails existen.
5	**Rol y estado del usuario se leen de la base en cada request**, no del token.	Desactivar o cambiar el rol de un usuario tiene efecto inmediato.
6	Bloqueo de login: **5 intentos fallidos por email en 15 min → 429**, contador en memoria.	§8. Alcanza para una sola instancia de la API por instalación (§16).
7	Endpoints `/users` (solo administrador) y `POST /me/password` .	No están en §9, pero hacen falta para HU01 y para la pantalla Configuración (§4).
8	**Nombre de cliente único sin distinguir mayúsculas** (índice único sobre `lower(name)`).	La importación CSV (§10) identifica al cliente por `client_name` : no puede haber ambigüedad.
9	`CHECK (due_date >= issue_date)` en facturas.	Evita datos incoherentes; no lo contradice ningún punto de la especificación.
10	El **saldo y el estado no se guardan**: se calculan en cada consulta (Python para mostrar, `CASE` en SQL para filtrar) con la fecha de hoy en `America/Argentina/Buenos_Aires` . Un test verifica que los dos cálculos coinciden.	§5 y HU05: la factura pasa a VENCIDA sin intervención manual.
11	Estado extra `ANULADA` , solo para mostrar facturas con `is_void` . Por defecto se excluyen de los listados.	§5 no define qué se muestra para una factura anulada.
12	**Anular una factura**: solo administrador, solo si no tiene pagos vigentes, y no se puede revertir. Una factura anulada no se puede editar.	Coherente con "anulación lógica de pago por administrador" (§4).
13	No se puede bajar el importe de una factura por debajo de lo ya cobrado. La edición bloquea la fila con `SELECT … FOR UPDATE` .	Evita saldos negativos y prepara la concurrencia con pagos del Sprint 2.
14	No se pueden cargar facturas a un **cliente inactivo**.	Uso típico de "desactivar".
15	**Montos en JSON como string** (`"125000.50"`).	Ningún cliente (escritorio/dashboard) los convierte a float por accidente.
16	Auditoría en `audit_events` dentro de la **misma transacción** que la operación, con el *diff* de los campos. Nunca se guardan contraseñas ni tokens.	§3: toda operación queda asociada al usuario autenticado.
17	Los tests usan una base aparte (`<POSTGRES_DB>_test`) que se crea y migra sola. Cada test corre en una transacción que se revierte.	Los datos de desarrollo no se tocan y los tests no dependen del orden.

Pendiente para próximos sprints

- Sprint 2: servicio de pagos (parcial/total, anulación con motivo, `FOR UPDATE` , auditoría).
- Sprint 5: si el bloqueo de login tiene que sobrevivir a reinicios o escalar a varias instancias, moverlo a la base.